

# 暑期实习生技术评估

方向一：机器人触觉感知与物理约束学习 | 机器人学 · 深度学习 · 物理仿真

## 关于本次评估

你好！我们有一个暑期实习机会，方向是**机器人触觉感知与物理约束学习**。

不需要有机器人研发的经验。如果你在**数学、物理和代码能力**上比较好，可以很容易完成我们在这个方向上的暑期实习适应力评估。

选一个你感兴趣的，尽快完成即可。它们从易到难：**物理刹车** → **指尖拼图** → **旋转不变**。

完成后把代码和说明发给我们。我们期待的**不是完美，而是清晰的思路和能跑通的结果**。

以下是三道技术评估题目的完整描述。请仔细阅读每道题目的**做什么**、**交付物**和**建议**三个部分，选择最适合你背景和兴趣的一道进行作答。

★ ★ ★

## 题目一：给神经网络加"物理刹车"

入门级

### 描述

机器人指尖预测接触力，但力不能为负，摩擦力也不能超过法向力的某个比例。你能否写一个"物理刹车层"，让神经网络的输出天生就遵守这些物理规则，而不是靠事后惩罚？

### 做什么

写一个 PyTorch 的 `nn.Module`，输入是网络原始输出的 `[法向力, 切向力_x, 切向力_y]`，输出是满足以下约束的"合法"力：

- 法向力  $\geq 0$
- 切向力大小  $\leq \mu \times \text{法向力}$  ( $\mu$  取 0.5)

这个层必须**可微分**（能反向传播），且能处理"法向力接近 0"这种危险边界。

### 交付物

- 一个 `.py` 文件，包含你的层 + 一段自动测试（随机输入 1000 次，检查 100% 满足约束，且梯度无 NaN）
- 200 字说明：你是怎么处理“法向力接近 0”时的数值稳定性的？

### 建议

对于题目缺失的上下文内容和数据，请你按照自己的理解去构造。我们欢迎你最大限度利用 AI。

## 题目二：两张图，拼出一个真相 进阶级

### 描述

指尖有两个传感器：一个摄像头拍到变形图像（ $64 \times 64$ ），一个压力传感器读到  $16 \times 16$  的矩阵。它们说的是同一次接触。你能让网络“看”懂这两张图，猜出 5 个数字吗？

### 做什么

1. 利用合成数据生成代码（物理启发的假数据，带噪声），生成训练集和测试集。
2. 搭一个简单网络：两个前端分别处理两张图，后面接一个简单融合层，预测 5 个数字：`[接触力, 接触面积, 压痕深度, 中心x, 中心y]`
3. 训练它，在测试集上跑出结果。

### 合成数据生成代码（Python）参考

```

import numpy as np
import torch

def generate_tactile_data(n_samples=1000):
    data = []
    for _ in range(n_samples):
        # 1. 采样物理状态
        F_n = np.random.uniform(0.1, 10.0)    # 法向力 (N)
        A = np.random.uniform(0.5, 5.0)       # 接触面积 (mm²)
        delta = np.random.uniform(0.1, 2.0)   # 压痕深度 (mm)
        x_c = np.random.uniform(8, 56)        # 接触中心 (pixel)
        y_c = np.random.uniform(8, 56)

        # 2. 生成视触觉位移场 (64x64, 2ch)
        xx, yy = np.meshgrid(np.arange(64), np.arange(64))
        dist = np.sqrt((xx - x_c)**2 + (yy - y_c)**2)
        sigma = np.sqrt(A) * 2

        disp_x = delta * np.exp(-dist**2 / (2*sigma**2)) * np.random.normal(1, 0.1)
        disp_y = delta * np.exp(-dist**2 / (2*sigma**2)) * np.random.normal(1, 0.1)
        tactile_img = np.stack([disp_x, disp_y], axis=0).astype(np.float32)

        # 3. 生成压阻矩阵 (16x16)
        xx_p, yy_p = np.meshgrid(np.linspace(0, 63, 16), np.linspace(0, 63, 16))
        dist_p = np.sqrt((xx_p - x_c)**2 + (yy_p - y_c)**2)
        sigma_p = sigma / 4
        pressure = F_n * np.exp(-dist_p**2 / (2*sigma_p**2))
        pressure += np.random.normal(0, 0.1 * F_n, pressure.shape)
        pressure = np.maximum(pressure, 0).astype(np.float32)

        data.append({
            'tactile_img': tactile_img,      # [2, 64, 64]
            'pressure_mat': pressure,        # [16, 16]
            'state': np.array([F_n, A, delta, x_c, y_c], dtype=np.float32)
        })
    return data

```

## 交付物

- 可运行的训练代码 + 测试集上的误差数字
- 300 字说明：如果你的模型明天要装到真实机器人上，你觉得最大的三个隐患是什么？

### 建议

对于题目缺失的上下文内容和数据，请你按照自己的理解去构造。我们欢迎你最大限度利用 AI。  
根据我们的参考代码，你可以自己改造更有挑战力的合成数据（譬如：数据缺失、数据噪声）。

## 题目三：转一转，它还能猜对吗？

挑战级

### 描述

你把指尖摄像头拍到的图像旋转  $90^\circ$ ，接触力的方向也应该转  $90^\circ$ ——这是物理常识。但普通神经网络（比如标准 CNN）并不“天生懂”这个常识。你能搭建一个懂旋转的网络，或者至少证明标准 CNN 有多笨吗？

### 做什么

1. 用合成数据生成“带方向标签”的触觉图像（比如一个椭圆接触区域，长轴方向 = 接触力方向）。
2. 实现一个旋转等变的网络（可以用 `escnn/e2cnn` 等现成库，也可以自己写简化版），要求：输入图像旋转后，输出的方向也同步旋转。
3. 与标准 CNN 做对比：譬如把测试图像旋转  $0^\circ$ 、 $90^\circ$ 、 $180^\circ$ 、 $270^\circ$ ，记录两种网络的预测误差。

### 交付物

- 对比实验代码 + 一张误差对比图（横轴：旋转角度，纵轴：方向预测误差）
- 300 字说明：如果机器人要抓一个从没见过的物体，这种“转一转还能猜对”的能力为什么重要？

### 建议

对于题目缺失的上下文内容和数据，请你按照自己的理解去构造。我们欢迎你最大限度利用 AI。